

Cosmic Daybook

Developer Technical Reference Manual

Version: March 2026

Platform:	iOS 27+ / macOS 27+
Framework:	SwiftUI + Core Data
Language:	Swift 6.0+
Sync:	CloudKit (optional)
AI:	Apple Intelligence (on-device + Private Cloud Compute)

Table of Contents

Part 1: Architecture Overview	6
What the App Is	6
High-Level Module Map	6
Architecture Pattern: MVVM + Services	6
Part 2: App Lifecycle	8
Entry Point	8
Synchronous Init (AppBootstrapping.performInitialSetup())	8
App Body	8
Async Bootstrap (.task modifier)	8
First Launch vs. Subsequent Launch	9
Bootstrap State Machine	9
Part 3: Navigation System	10
AppRouter	10
Sheet Destinations	10
Root Navigation Items	10
Navigation groups	11
Special Routing	11
RootView	11
Layout Structure	11
Platform Differences	11
State Persistence	12
Quick Action System	12
QuickNoteGlassButton	12
Pie Menu	12
Command Bar	13
Part 4: Dependency Injection	15
AppDependencies	15
Lazy Initialization Pattern	15
Service Groups	15

Key Services Available	15
Part 5: Data Layer	17
CloudKit Compatibility Rules	17
Rule 1: String UUIDs, Not UUID Type	17
Rule 2: Raw String Enums	17
Rule 3: No Unique Constraints	17
Rule 4: External Storage for Large Data	17
Rule 5: String Foreign Keys, Not Core Data Relationships	17
Rule 6: modifiedAt for Conflict Resolution	18
Core Entities (60 Core Data entities)	18
CDStudent	18
CDLesson	18
CDLessonAssignment (Presentation)	19
CDWorkModel	20
CDNote	21
Model Relationship Map	22
Other Entities (Quick Reference)	23
Part 6: Repository Pattern	25
Repository Protocol	25
Typical Repository Methods	25
LessonRepository Special Methods	25
SaveCoordinator	25
Key Behaviors	26
Usage	26
Error Handling Flow	26
Part 7: Service Layer	27
Service Patterns	27
Key Services Deep Dive	27
LifecycleService	27
ChatService + AIClientRouter	27
ReminderSyncService (macOS)	28
CloudKitSyncStatusService	28

EnhancedBackupService	28
ToastService	29
Part 8: Feature Modules	30
Today Hub	30
TodayViewModel Outputs	30
Supporting Components	30
Reload Strategy	31
Students Module	31
Student Roster	31
Student Detail View	31
AI Analysis	31
Lessons Module	31
Lesson Library	31
Lesson Formats	32
Import/Export	32
Work Module	32
Work Creation Sources	32
Work Check-Ins	33
Multi-Student Work	33
Practice Sessions	33
PDF Rendering	33
Presentations Module	33
Scheduling Calendar	33
State Machine	33
Mastery Tracking	33
Attendance Module	34
Planning Module	34
Notes Module	34
Todos Module	35
Settings Module	35
Backup & Restore	35

AI Chat	36
Part 9: Utilities & Extensions	37
Safe Fetch Pattern	37
Collection Extensions	37
Date Utilities	37
String Utilities	37
Other Utilities	38
Part 10: How-To Reference	39
Adding a New Entity	39
Creating a Repository	39
Adding a Service	40
Building a New Screen	41
Adding a Command Bar Intent	42
CloudKit Considerations Checklist	42
Part 11: Configuration & Debugging	43
CloudKit Configuration	43
EventKit Permissions (macOS)	43
API Keys	43
Performance Monitoring	43
Debugging Aids	43
Build Configuration	44
Code Standards	44
Appendix A: File Directory Map	45
Appendix B: Glossary	47

Part 1: Architecture Overview

What the App Is

Cosmic Daybook is a Montessori teacher's all-in-one classroom management app. It tracks students, lessons, presentations, student work, attendance, observations, scheduling, curriculum progression, and more. It runs natively on iOS and macOS using SwiftUI, with Core Data (NSPersistentCloudKitContainer) for persistence and optional CloudKit for iCloud sync.

High-Level Module Map

```
CosmicDaybookApp (entry point)
|
v
AppBootstrapper (startup state machine)
|
v
RootView (main shell)
|
+-- AppRouter (navigation coordinator)
+-- AppDependencies (dependency injection)
|
|   +-- Services
|   |   +-- LifecycleService
|   |   +-- ChatService + AIClientRouter
|   |   +-- ReminderSyncService
|   |   +-- CloudKitSyncStatusService
|   |   +-- BackupService
|   |   +-- ToastService
|   |   +-- ... (70+ services)
|   |
|   +-- Coordinators
|   |   +-- SaveCoordinator
|   |   +-- RestoreCoordinator
|   |
+-- Feature Views
    +-- TodayView (daily dashboard)
    +-- StudentsListView / StudentDetailView
    +-- LessonsListView / LessonDetailView
    +-- WorkListView / WorkDetailView
    +-- AttendanceView
    +-- PlanningViews (agenda, checklist, projects)
    +-- InboxView
    +-- SettingsView
    +-- ... (40+ feature views)
```

Architecture Pattern: MVVM + Services

The app follows a layered MVVM architecture with a service layer:

```
SwiftUI View
  | Binds to (via @Observable or @FetchRequest)
  v
ViewModel (@Observable, @MainActor)
  | Calls methods on
  v
Service (business logic, @MainActor)
  | Uses for data access
  v
Repository (type-safe CRUD, uses SaveCoordinator)
  | Reads/writes via
  v
NSManagedObjectContext
  | Syncs via (optional)
  v
CloudKit (iCloud)
```

When to use each layer:

- **View with @FetchRequest:** Simple list screens where the data maps directly to what's displayed. No transformation needed.
- **View with ViewModel:** Screens that combine multiple data sources, compute derived state, or need complex filtering/sorting. Examples: TodayView, GiveLessonViewModel.
- **Service:** Reusable business logic called from multiple ViewModels or other services. Example: LifecycleService (work state transitions).
- **Repository:** Type-safe CRUD operations for a single entity. Wraps Core Data queries with error handling and SaveCoordinator integration.

Part 2: App Lifecycle

Entry Point

File: `AppCore/CosmicDaybookApp.swift`

The `@main` struct `CosmicDaybookApp` conforms to `App`. Its `init()` runs synchronously before any UI renders.

Synchronous Init (`AppBootstrapping.performInitialSetup()`)

This runs first and handles:

1. Registers default UserDefaults (CloudKit sync enabled by default)
2. Disables CloudKit during XCTest runs
3. Starts performance monitoring (stutter detection for frames >100ms)
4. Configures SQLite environment variables for Debug builds
5. Initializes `SaveCoordinator.shared` and `RestoreCoordinator.shared`

App Body

The body provides:

- A `WindowGroup` with the main content
- On macOS, additional `Window` groups for detail views (Work, Student, Lesson windows opened via `openWindow`)
- The Core Data stack (`NSPersistentCloudKitContainer`) is injected as an environment object
- `SaveCoordinator`, `RestoreCoordinator`, `ErrorCoordinator`, `AppRouter`, and `AppDependencies` are injected into the environment

Async Bootstrap (`.task` modifier)

After the UI appears (showing a splash screen), the `.task` block runs:

```

AppBootstrapper.bootstrap(coreDataStack:)
|
|-- State: .initializingContainer
|   |-- Set up AppCalendar with correct timezone
|   |-- Migrate lesson files to iCloud Drive (one-time)
|
|-- State: .migrating
|   |-- Run quick migrations (attendance types, GroupTrack defaults)
|   |-- Initialize ReminderSyncService (macOS only)
|   |-- Register for CloudKit remote notifications
|
|-- State: .ready (UI unblocks here)
|
|-- Background Task (off main thread):
|   |-- Data normalization
|   |-- Relationship backfills
|   |-- Deduplication (CloudKit conflict cleanup)
|   |-- Integrity repairs (probabilistic, ~10% of launches)

```

First Launch vs. Subsequent Launch

Step	First Launch	Subsequent Launch
<code>performInitialSetup()</code>	Registers all defaults	No-op (defaults exist)
Onboarding check	Shows <code>OnboardingView</code>	Skips to <code>RootView</code>
Lesson file migration	Migrates local files to iCloud	Skips (flag set)
Quick migrations	Runs all	Skips completed ones (flags)
Heavy migrations	Runs all	Runs only if needed
CloudKit setup	First sync pulls remote data	Incremental sync

Bootstrap State Machine

```

enum BootstrapState {
    case idle
    case initializingContainer
    case migrating
    case ready
}

```

The splash screen checks `bootstrapper.state != .ready` to decide whether to show the loading UI.

Part 3: Navigation System

AppRouter

File: `AppCore/AppRouter.swift`

`AppRouter` is an `@Observable @MainActor` singleton that coordinates all navigation. Views access it via `@Environment(\.appRouter)`.

Sheet Destinations

```
enum NavigationDestination: Identifiable {
    case newLesson(defaultSubject, defaultGroup)
    case importLessons
    case importStudents
    case newStudent
    case createBackup
    case restoreBackup
    case newWork
    case openAttendance
    case openStudentDetail(UUID)
    case quickActions
}
```

A view presents sheets by setting `appRouter.navigationDestination`.

Root Navigation Items

```
// AppCore/RootView+NavigationItem.swift
enum NavigationItem: String, Hashable, Identifiable, CaseIterable {
    case today, todos
    case students, attendance, meetings, parentReports, progressDashboard
    case planningAgenda, lessons
    case planningChecklist, curriculumMap, planningCalendar, smallSequencePlanner
    case logs, notes
    case teachingAlbums, stories, bookClub, procedures, resourceLibrary, supplies, goingOut,
        community, schedules, thisWeeksParsha, parshaCalendar, lessonRecall, planningProjects
    case askAI, settings
    case note, more, perpetualCalendar // aliases: kept so a saved raw value still decodes
}
```

Navigation happens by setting `appRouter.selectedNavItem = .students`.

Raw values are frozen. They live in `@SceneStorage("RootView.selectedNavItem")`, so renaming one silently resets every device's selection. `NavigationGroupTests.rawValuesArePinned` holds every literal; a case that stops being a destination stays in the enum and is mapped by `NavigationItem.aliases` → `canonical` (`.perpetualCalendar` → `.planningCalendar`, `.note/.more` → `.today`). A case that is removed outright is handled by `NavigationSelectionRestorer` (same file), the pure function that turns the saved raw value, the pre-`NavigationItem` `Tab` string, and the old Planning mode into today's destination.

Navigation groups

`RootView.NavigationGroup` (`AppCore/RootView+NavigationGroup.swift`) is the single table that decides where every destination sits. The macOS sidebar (`RootSidebar`), the iPad sidebar and the iPhone tab bar plus its More list (`RootAdaptiveTabs`) all iterate it; nothing lists items by hand. Groups in order: Today, Children, Lessons & Work, Planning, Records, Classroom, Library (those two collapsed by default), System. `NavigationGroup.primaryTabs` is the iPhone bar — Today, Students, Attendance, Lessons & Work — and `secondaryGroups` is what "More" shows. `NavigationGroupTests` pins that every destination is in exactly one group, that the `TabView` lists each once, and the group order.

The macOS sidebar persists each group's collapsed state under `UserDefaultsKeys.sidebarGroupExpanded(groupID)` and auto-expands the group holding a selection that arrives from outside the sidebar (a Go-menu shortcut, an album deep link, a restored selection), so nothing lands invisibly inside a collapsed Classroom or Library.

Special Routing

- **Plan Lesson Request:** `appRouter.planLessonRequest = PlanLessonRequest(studentID: id, date: date)` — triggers the lesson planning flow with pre-filled context.
- **Restore Signals:** `appDataWillBeReplaced` / `appDataDidRestore` — used to show a blocking overlay during backup restoration.
- **Inbox Refresh:** `planningInboxRefreshTrigger` — a UUID that changes to signal the inbox to reload.

RootView

File: `AppCore/RootView.swift` (split across multiple files)

Layout Structure

```
RootView
+-- Warning Banners (restore in progress, sync issues)
+-- Divider
+-- Main Content Area
|   +-- Sidebar (RootSidebar on macOS/visionOS, RootAdaptiveTabs on iOS) – rows from
RootView.NavigationGroup
|   +-- Detail (RootDetailContent) – routes NavigationItem.canonical to the right view
+-- Overlays
    +-- QuickNoteGlassButton (floating action button)
    +-- TipView (TipKit feature hints)
    +-- Sheet presentations (new lesson, new student, etc.)
```

Platform Differences

Feature	iOS	macOS
Layout	<code>TabView(.sidebarAdaptable)</code> — tab bar on iPhone, sidebar on iPad	<code>NavigationSplitView</code>
Detail windows	<code>Sheets</code>	Separate windows via <code>openWindow</code>
Sidebar	iPhone: Today, Students, Attendance, Lessons & Work, More; iPad: grouped sidebar	Grouped sidebar with persisted collapse state

State Persistence

- `@SceneStorage("selectedNavItem")` — persists the selected tab across app launches
- `@AppStorage` — user preferences (test student visibility, agenda ordering)

Quick Action System

QuickNoteGlassButton

File: `AppCore/RootView/QuickNoteGlassButton.swift`

A floating quick-capture button. Shown unless the guide turns it off (Settings ■ General, or View ■ Show Quick Capture Button on the Mac; `UserDefaultsKeys.quickCaptureButtonVisible`).

Gesture	Action
Tap	Opens the Command Bar (quick capture)
Long press (400ms), slide, release	Expands the Pie Menu and runs the segment released on
Right-click (macOS)	Lists the five create actions
Drag	Repositions the button (saved in UserDefaults)

Pie Menu

File: `AppCore/RootView/PieMenu.swift`

Five actions arranged in a circle (radius: 95pt):

Action	Color	What it does
New Presentation	Blue	Creates a draft presentation
New Work Item	Orange	Opens work creation

Action	Color	What it does
Record Practice	Pink	Opens practice recording
New Todo	Green	Creates a todo
New Note	Purple	Opens note creation

Command Bar

Files: `Services/CommandBar/CommandBarTypes.swift`,
`Services/CommandBar/LocalCommandParser.swift`,
`Components/CommandBar/CommandBarSheet.swift`, `ViewModels/CommandBarViewModel.swift`

The command bar accepts natural language input and parses it into structured actions.

Intent Detection Keywords:

Intent	Trigger Words
Record Presentation	"gave", "presented", "showed", "demonstrated"
Assign Work	"assign", "assigned", "work"
Record Practice	"practice", "practiced", "practicing"
Add Note	"note", "observe", "noticed", "saw"
Add Todo	"todo", "remind", "task"

Parsing Pipeline:

```

User input: "gave Sophia long division"
  |
  v
1. Detect intent via keyword → recordPresentation
2. Extract student names via StudentTagger → [Sophia]
3. Fuzzy-match lesson name → "Long Division" (lessonID)
4. Extract remaining text → (none)
5. Compute confidence → 0.8
  |
  v
Output: ParsedCommand {
  intent: .recordPresentation,
  studentIDs: [sophia_uuid],
  lessonID: long_division_uuid,
  confidence: 0.8
}

```

Confidence Scoring:

- Base: intent keyword confidence
- +0.2 if students matched
- +0.2 if lesson matched
- +0.1 bonus for short input (12 words or fewer)

-
- Capped at 1.0

The parser runs on an actor (`LocalCommandParser`) to keep the main thread free.

Part 4: Dependency Injection

AppDependencies

File: `AppCore/AppDependencies.swift`

An `@Observable @MainActor` class that serves as the central dependency container. Views access it via:

```
@Environment(\.dependencies) private var dependencies
```

Lazy Initialization Pattern

Services are created on first access, not at startup:

```
private var _toastService: ToastService?
var toastService: ToastService {
    if _toastService == nil {
        _toastService = ToastService()
    }
    return _toastService!
}
```

This reduces launch time since most services aren't needed immediately.

Service Groups

The container is split across extension files for organization:

File	Services
<code>AppDependencies.swift</code>	Core: repositories, toast, router, memory monitor, sync, command bar
<code>AppDependencies+AIServices.swift</code>	AIClientRouter, ChatService, StudentAnalysisService, ReportGeneratorService
<code>AppDependencies+BackupServices.swift</code>	BackupService, EnhancedBackupService, RestoreCoordinator, CloudBackupService

Key Services Available

```
dependencies.toastService      // Toast notifications
dependencies.appRouter         // Navigation
dependencies.reminderSync     // EventKit sync (macOS)
dependencies.calendarSync     // Calendar display
dependencies.groupTrackService // Track management
dependencies.trackProgressResolver // Progress calculation
dependencies.memoryPressureMonitor // Cache management
dependencies.commandBarService // Command bar
dependencies.dataQueryService  // Complex queries
```

Part 5: Data Layer

CloudKit Compatibility Rules

Every entity in the app must follow these rules for CloudKit sync compatibility:

Rule 1: String UUIDs, Not UUID Type

```
// CORRECT – String attribute in Core Data model
@NSManaged public var studentID: String

// WRONG - CloudKit can't handle UUID type reliably
@NSManaged public var studentID: UUID
```

All foreign key references use `String` to store UUID values. Convert with `UUID(uuidString: studentID)` when needed.

Rule 2: Raw String Enums

```
// CORRECT
@NSManaged public var statusRaw: String
var status: WorkStatus {
    get { WorkStatus(rawValue: statusRaw) ?? .active }
    set { statusRaw = newValue.rawValue }
}

// WRONG - CloudKit can't handle enums directly
// (no enum attribute type in Core Data model editor)
```

The pattern is: store a `*Raw: String` property, provide a computed property for type-safe access.

Rule 3: No Unique Constraints

CloudKit creates duplicate records during merge conflicts. Unique constraints would crash. Instead, the app uses a deduplication service that runs periodically to clean up duplicates.

Rule 4: External Storage for Large Data

In the Core Data model editor, enable "Allows External Storage" for Binary Data attributes.

```
@NSManaged public var pagesFileBookmark: Data?
```

Data larger than ~100KB should use external storage to avoid bloating the SQLite database.

Rule 5: String Foreign Keys, Not Core Data Relationships

```
// CORRECT - resilient to sync ordering
@NSManaged public var studentID: String

// RISKY - can orphan if student syncs after work item
// @NSManaged public var student: CDStudent? (relationship)
```

Relationships are resolved at query time, not enforced at the schema level. This prevents orphaning when CloudKit syncs records in unpredictable order.

Rule 6: modifiedAt for Conflict Resolution

Every entity includes:

```
@NSManaged public var modifiedAt: Date
```

This timestamp is used for last-writer-wins conflict resolution during CloudKit sync.

Core Entities (60 Core Data entities)

CDStudent

File: `Students/Models/StudentEntity.swift`

Field	Type	Purpose
<code>id</code>	<code>UUID</code>	Primary identifier
<code>firstName, lastName</code>	<code>String</code>	Name fields
<code>nickname</code>	<code>String</code>	Optional display name
<code>birthday</code>	<code>Date</code>	Birth date
<code>levelRaw</code>	<code>String</code>	<code>.lower</code> or <code>.upper</code> elementary
<code>enrollmentStatusRaw</code>	<code>String</code>	<code>.enrolled</code> or <code>.withdrawn</code>
<code>nextLessons</code>	<code>[String]</code>	Ordered list of upcoming lesson UUIDs
<code>dateStarted, dateWithdrawn</code>	<code>Date?</code>	Enrollment dates
<code>manualOrder</code>	<code>Int</code>	Custom sort position
<code>modifiedAt</code>	<code>Date</code>	CloudKit conflict resolution

Indexes: `[levelRaw, manualOrder, modifiedAt, enrollmentStatusRaw]`

Inverse Relationships: `documents: [CDDocument]?`

CDLesson

File: `Lessons/LessonEntity.swift`

Field	Type	Purpose
<code>id</code>	<code>UUID</code>	Primary identifier
<code>name</code>	<code>String</code>	Lesson title
<code>subject, group</code>	<code>String</code>	Curriculum organization
<code>orderInGroup, sortIndex</code>	<code>Int</code>	Display ordering
<code>subheading</code>	<code>String</code>	Brief description
<code>writeUp</code>	<code>String</code>	Full presentation script
<code>materials</code>	<code>String</code>	Required materials
<code>purpose</code>	<code>String</code>	Educational purpose
<code>ageRange</code>	<code>String</code>	Target age range
<code>teacherNotes</code>	<code>String</code>	Personal notes
<code>suggestedFollowUpWork</code>	<code>String</code>	Follow-up activity ideas
<code>prerequisiteLessonIDs</code>	<code>String</code>	Comma-separated prerequisite UUIDs
<code>relatedLessonIDs</code>	<code>String</code>	Comma-separated related UUIDs
<code>greatLessonRaw</code>	<code>String?</code>	Cosmic education connection
<code>lessonFormatRaw</code>	<code>String</code>	" <code>standard</code> " or " <code>story</code> "
<code>parentStoryID</code>	<code>String?</code>	Parent story UUID for branching
<code>sourceRaw</code>	<code>String</code>	" <code>album</code> " or " <code>personal</code> "
<code>defaultWorkKindRaw</code>	<code>String?</code>	Preferred work type when assigned
<code>pagesFileBookmark</code>	<code>Data?</code>	External file reference (external storage)
<code>primaryAttachmentID</code>	<code>String?</code>	Primary attachment

Indexes: [`subject, sortIndex`], [`name`]

CDLessonAssignment (Presentation)

File: `Models/Presentation.swift`

This is the central scheduling and recording model. It represents the act of presenting a lesson to students.

Field	Type	Purpose
<code>id</code>	<code>UUID</code>	Primary identifier

Field	Type	Purpose
<code>stateRaw</code>	<code>String</code>	State machine: <code>draft</code> , <code>scheduled</code> , <code>presented</code>
<code>scheduledFor</code>	<code>Date?</code>	When to present (nil for drafts)
<code>scheduledForDay</code>	<code>Date</code>	Start-of-day (denormalized for efficient date queries)
<code>presentedAt</code>	<code>Date?</code>	When actually presented (immutable record)
<code>lessonID</code>	<code>String</code>	Lesson UUID (indexed)
<code>lessonTitleSnapshot</code>	<code>String?</code>	Frozen lesson title at presentation time
<code>lessonSubheadingSnapshot</code>	<code>String?</code>	Frozen lesson subheading
<code>_studentIDsData</code>	<code>Data</code>	JSON-encoded student UUID array
<code>needsPractice</code>	<code>Bool</code>	Flag: student should practice
<code>needsAnotherPresentation</code>	<code>Bool</code>	Flag: re-present later
<code>followUpWork</code>	<code>String</code>	Follow-up work description
<code>notes</code>	<code>String</code>	Presentation notes
<code>manuallyUnblocked</code>	<code>Bool</code>	Override prerequisite checking
<code>trackID, trackStepID</code>	<code>String?</code>	Curriculum track context

State Machine:

```

draft ■■> scheduled ■■> presented
|
| (immutable after |
|   this point)   |
+■■> presented    |
    (skip scheduling) |

```

Once a presentation reaches `presented`, the `lessonTitleSnapshot` and `lessonSubheadingSnapshot` are frozen — even if the lesson is later edited, the historical record is preserved.

Indexes: [`stateRaw`], [`scheduledForDay`], [`presentedAt`], [`lessonID`]

CDWorkModel

File: `Work/Models/WorkModelEntity.swift`

Field	Type	Purpose
<code>id</code>	<code>UUID</code>	Primary identifier

Field	Type	Purpose
<code>title</code>	<code>String</code>	Work item name
<code>statusRaw</code>	<code>String</code>	<code>active</code> , <code>review</code> , <code>complete</code>
<code>kindRaw</code>	<code>String</code>	<code>practice</code> , <code>followUp</code> , <code>research</code> , <code>report</code>
<code>studentID</code>	<code>String</code>	Primary student UUID (indexed)
<code>lessonID</code>	<code>String</code>	Related lesson UUID
<code>presentationID</code>	<code>String?</code>	Originating presentation
<code>trackID</code> , <code>trackStepID</code>	<code>String?</code>	Track context
<code>dueAt</code>	<code>Date?</code>	Due date (indexed)
<code>assignedAt</code>	<code>Date</code>	When assigned
<code>completedAt</code>	<code>Date?</code>	When completed
<code>lastTouchedAt</code>	<code>Date</code>	Last interaction
<code>completionOutcomeRaw</code>	<code>String?</code>	Mastered, needs review, etc.
<code>scheduledNote</code>	<code>String?</code>	Scheduling notes
<code>sourceContextTypeRaw</code>	<code>String?</code>	Origin type (e.g., project session)
<code>sourceContextID</code>	<code>String?</code>	Origin ID

Relationships:

- `participants`: `[CDWorkParticipant]` — individual student progress in group work
- `checkIns`: `[CDWorkCheckIn]` — progress check-in records
- `steps`: `[CDWorkStep]` — step-by-step breakdown
- `unifiedNotes`: `[CDNote]` — attached observations

Compound Indexes: `[studentID, statusRaw]`, `[statusRaw, dueAt]`, `[presentationID, completedAt]`

Work Lifecycle:

```

active ■■> review ■■> complete
|
| (teacher reviews) +-- completionOutcome recorded
|                   +-- completedAt set
+■■> complete       +-- track progress updated
  (skip review)

```

CDNote

File: `Models/NoteEntity.swift`

Field	Type	Purpose	
id	UUID	Primary identifier	
body	String	Note content	
createdAt	Date	When created (indexed)	
updatedAt	Date	Last modified	
isPinned	Bool	Pin to top of lists	
includeInReport	Bool	Include in progress reports	
needsFollowUp	Bool	Flag for action needed	
imagePath	String?	Attached image path	
categoryRaw	String	Legacy category (migration)	
tags	[String]	Modern tags, format: `\"Name\\`	Color\"`
reportedBy	String?	Reporter type (e.g., \"guide\")	
reporterName	String?	Reporter name	
scopeIsAll	Bool	Indexed: true if scope is <code>.all</code>	
searchIndexStudentID	String?	First student ID for search	

Scope System:

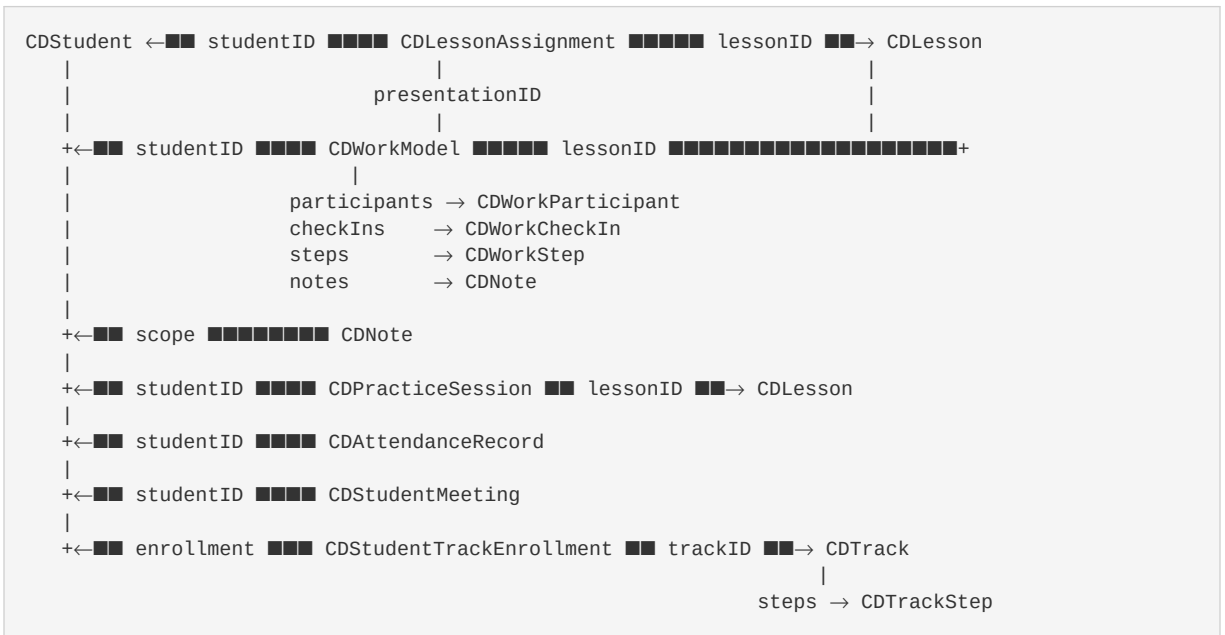
Notes use a flexible scoping system stored as JSON:

```
enum NoteScope: Codable {
    case all           // Applies to all students
    case student(UUID) // Single student
    case students([UUID]) // Multiple students
}
```

Multi-student notes use a junction table (`CDNoteStudentLink`) to enable efficient per-student queries.

Indexes: `[createdAt]`, `[searchIndexStudentID]`, `[scopeIsAll]`

Model Relationship Map



Other Entities (Quick Reference)

Entity	Purpose
CDPracticeSession	Records practice with quality, duration, method
CDAttendanceRecord	Daily present/tardy/absent per student
CD_TODOItem	Teacher tasks with subtasks, due dates, tags
CD_TODOSubtask	Checklist items within a todo
CDReminder	EventKit-synced reminders
CDSchedule / CDScheduleSlot	Weekly schedule templates
CDCalendarEvent	Calendar integration events
CDNonSchoolDay / CDSchoolDayOverride	Calendar exceptions
CDStudentMeeting	One-on-one meetings with outcomes
CDMeetingTemplate	Reusable meeting structures
CDProject / CDProjectSession	Project-based learning
CDTrack / CDTrackStep	Curriculum progression paths
CDGroupTrack	Group variations of tracks
CDStudentTrackEnrollment	Student enrollment in tracks
CDSupply / CDSupplyTransaction	Classroom inventory
CDCommunityTopic / CDProposedSolution	Community discussions

Entity	Purpose
CDDocument	Learning resources
CDIssue / CDIssueAction	Issue tracking
CDProcedure	Standard procedures
CDNoteTemplate	Reusable observation templates
CDLessonAttachment	Files attached to lessons
CDLessonPresentation	Per-student mastery tracking
CDAlbumGroupOrder / CDAlbumGroupUIState	UI state persistence
CDTodayAgendaOrder	Custom agenda ordering
CDDevelopmentSnapshot	Progress snapshots
CDPlanningRecommendation	AI recommendations
CDWorkCompletionRecord	Historical completion tracking

Part 6: Repository Pattern

Repositories are small structs a view or service constructs directly with the context it already holds (for example `StudentRepository(context:saveCoordinator:)`). There is no central container; a repository exists only where a feature calls it.

Repository Protocol

Repositories conform to `SavingRepository`:

```
protocol SavingRepository {
    associatedtype Entity: NSManagedObject
    var context: NSManagedObjectContext { get }
    var saveCoordinator: SaveCoordinator? { get }
}
```

Typical Repository Methods

Using `StudentRepository` as an example:

```
struct StudentRepository: SavingRepository {
    // Fetch
    func fetchStudent(id: UUID) -> CDStudent?
    func fetchStudents(predicate:, sortBy:) -> [CDStudent]

    // Create
    func createStudent(firstName:, lastName:, ...) -> CDStudent

    // Update
    func updateStudent(id:, firstName:, ...) -> Bool

    // Delete
    func deleteStudent(id:) -> Bool

    // Save (delegates to coordinator)
    func save(reason: String?) -> Bool
}
```

LessonRepository Special Methods

```
// Story format support
func fetchRootStories() -> [CDLesson]
func fetchChildStories(parentID: String) -> [CDLesson]
```

SaveCoordinator

File: `Backup/SaveCoordinator.swift`

Centralizes all save operations with batching and error handling.

Key Behaviors

Feature	Detail
Debounce	500ms — coalesces multiple saves
Error capture	Stores last error message for UI display
Toast integration	Notifies user of save failures
Reason tracking	Optional reason string for debugging
Suppress alerts	Can silence alerts during migrations

Usage

```
// Direct save
saveCoordinator.save(context, reason: "Updated work progress")

// Scheduled save (debounced)
saveCoordinator.scheduleSave(context, reason: "Auto-save after edit")
```

Error Handling Flow

```
1. Check context.hasChanges
2. Attempt context.save()
3. On failure:
  a. Extract NSError with underlying errors
  b. Append reason string if provided
  c. Store in lastErrorMessage
  d. Set showAlert = true (unless suppressed)
  e. Post toast notification
```

Part 7: Service Layer

Service Patterns

Services in this app follow these conventions:

- `@MainActor` — ensures thread safety for UI-related operations
- **struct for stateless services** — e.g., `LifecycleService` with static methods
- **final class for stateful services** — e.g., `ChatService` with session state
- **Protocol-based design** — enables testing (e.g., `MCPCClientProtocol` for API client)
- **Registered in `AppDependencies`** — lazy-initialized on first access

Key Services Deep Dive

LifecycleService

File: `Work/Services/LifecycleService.swift`

Manages work item state transitions and data integrity.

Key Methods:

Method	Purpose
<code>safeFetch(_:using:)</code>	Returns empty array on error instead of throwing
<code>fetchWorkModel(presentationID:)</code>	Single fetch with <code>fetchLimit = 1</code>
<code>fetchAllWorkModels(presentationID:)</code>	Batch fetch by presentation
<code>upsertLessonPresentation(...)</code>	Idempotent create-or-update
<code>cleanOrphanedStudentIDs()</code>	Removes references to deleted students

Performance: Uses predicates and fetch limits to minimize in-memory work. Never loads all records when only one is needed.

ChatService + AIClientRouter

Files: `Chat/Services/ChatService.swift`, `Services/AI/AIClientRouter.swift`

AIClientRouter

Conforms to `MCPCClientProtocol` and sends every request to Apple Intelligence: the on-device model first, then Private Cloud Compute when Settings → AI allows it. See [Documentation/Architecture/AI.md](#).

ChatService

Orchestrates AI chat sessions with classroom context.

Session Flow:

```
1. startSession() – build classroom snapshot (student names, lesson stats)
2. User sends message
3. Refresh snapshot if stale
4. Build question-specific context (Tier 2 enrichment)
5. Assemble system message with classroom context
6. Trim message history to token budget (max 10 messages)
7. Call API (temperature: 0.7, max tokens: 2048)
8. Hand the answer to the UI via the onText callback (the whole answer so far;
   StreamingTextThrottle shows at most ~10 updates a second)
```

ReminderSyncService (macOS)

Purpose: Two-way sync between the app's `CDReminder` entity and macOS Reminders (EventKit).

- Creates/updates/deletes reminders in both directions
- Respects EventKit permissions
- Handles conflict resolution via timestamps
- Only available on macOS (conditional compilation)

CloudKitSyncStatusService

File: `Services/CloudKitSyncStatusService.swift`

Monitors iCloud sync health and exposes observable state.

Observable Properties:

```
var isSyncing: Bool
var lastSuccessfulSync: Date?
var lastSyncError: String?
var syncHealth: SyncHealth
var isNetworkAvailable: Bool
var isiCloudAvailable: Bool
var pendingLocalChanges: Int
```

Startup Behavior:

- 15-second grace period — prevents false "offline" reports
- Observer setup delayed 2 seconds for CloudKit stabilization
- Persists last sync date in UserDefaults

Sub-Services:

- `NetworkMonitoring` — network reachability
- `SyncRetryLogic` — retry management
- `CloudKitHealthCheck` — health assessment

EnhancedBackupService

File: `Backup/EnhancedBackupService.swift`

Full database backup and restore with multiple strategies.

Export Modes:

Mode	Description
Standard	Original format (backward compatible)
Streaming	Recommended — better memory usage for large databases
Incremental	Only changed data since last backup

Features:

- Optional password encryption
- Progress callbacks for UI
- Size estimation before export
- Automatic post-export verification
- Cloud sync conflict detection

Export Flow:

```
exportBackup(context, to: url, password:, mode:, progress:)
|
+-- Estimate backup size
+-- Export data (streaming/incremental/standard)
+-- Verify integrity if enabled
+-- Return EnhancedBackupOperationSummary
```

ToastService

Purpose: App-wide toast notifications with auto-dismiss.

```
toastService.show("Work completed", style: .success)
toastService.show("Save failed", style: .error)
```

Toasts auto-dismiss after a configurable delay and support stacking.

Part 8: Feature Modules

Today Hub

Files: `Today/Views/`, `Today/ViewModels/TodayViewModel.swift`, `Today/Support/`

The daily dashboard. Shows everything relevant to the selected date.

TodayViewModel Outputs

Property	What it shows
<code>todaysLessons</code>	Presentations scheduled for selected date
<code>overdueSchedule</code>	Work items past their due date
<code>todaysSchedule</code>	Work items due today
<code>staleFollowUps</code>	Follow-ups that haven't been addressed
<code>agendaItems</code>	Unified, user-orderable agenda
<code>completedWork</code>	Finished work items
<code>todaysReminders</code>	Reminders due today
<code>overdueReminders</code>	Past-due reminders
<code>anytimeReminders</code>	Reminders with no due date
<code>todaysCalendarEvents</code>	Calendar events
<code>scheduledMeetings</code> / <code>completedMeetings</code>	Meetings
<code>attendanceSummary</code>	Attendance stats
<code>recentNotes</code>	Recent observations

Supporting Components

- **TodayCacheManager** — lookup dictionaries (`studentsByID`, `lessonsByID`, `workByID`) to avoid per-row database queries
- **TodayDataFetcher** — batches related database queries
- **TodayScheduleBuilder** — constructs the schedule from work data
- **TodayAttendanceLoader** — fetches and summarizes attendance
- **SchoolCalendarService** — the app-wide school-day cache (`AppCore/`), kept per persistent store coordinator so Sample Class and My Class never answer for each other

Reload Strategy

- Debounced reloads when date or level filter changes
- Clears recent notes cache on date change to prevent unbounded memory growth

Students Module

Directory: [Students/](#)

The feature is organized by subdomain: [Roster/](#), [Detail/](#), [Models/](#), [Files/](#), [Import/](#), [Notes/](#), [Presentations/](#), [Progress/](#), [Selection/](#), [Printing/](#), [Meetings/](#), [Insights/](#), [Recall/](#), [YearPlan/](#), and [LessonDetail/](#).

Student Roster

- List view with filtering by level and enrollment status
- Sorting by name, manual order, or level
- Search by name (fuzzy matching)

Student Detail View

Tabbed interface:

Tab	Content
Presentations	Lesson history with mastery state per lesson
Notes	Observation timeline with filters
Files	Attached documents
History	Complete record across all models
Meetings	One-on-one meeting records
Tracks	Curriculum progression progress

AI Analysis

[StudentAnalysisService](#) sends student data to Apple Intelligence (through [AIClientRouter](#)) for analysis — progress trends, patterns, recommendations.

Lessons Module

Directory: [Lessons/](#)

Lesson Library

Organized by subject and group. Each lesson detail view shows:

- Presentation script (writeUp)
- Materials, purpose, age range
- Teacher notes
- Prerequisite and related lessons (linked)
- Attachments and sample work
- Per-student progress (who's been presented, who's practicing, who's proficient)

Lesson Formats

Format	Description
Standard	Traditional Montessori lesson
Story	Parent-child branching — a main story with chapter offshoots

Story lessons use `parentStoryID` to create a tree structure. `fetchRootStories()` gets top-level stories; `fetchChildStories(parentID:)` gets branches.

Import/Export

CSV import and export for bulk lesson management. `LessonFileStorage` handles file attachments with iCloud Drive integration.

Work Module

Directory: `Work/`

The largest module. Manages the complete lifecycle of student work items.

The feature is organized into `Agenda/`, `CheckIns/`, `Completion/`, `Detail/`, `Models/`, `Practice/`, `SampleWork/`, `Steps/`, `Support/`, `Services/`, and `WorkCard/`.

Work Creation Sources

Source	How work is created
Presentation	Automatically when a lesson is presented with follow-up work
Manual	Teacher creates directly
Inbox	From follow-up recommendations
Project	From project session assignments
Track	From track step completion

Work Check-Ins

`CDWorkCheckIn` records periodic progress notes. Check-in style can be:

- **Individual** — per-student in group work
- **Group** — single check-in for all participants
- **Flexible** — either style

Multi-Student Work

`CDWorkParticipant` tracks individual student progress within a group work item. Each participant can have their own completion state.

Practice Sessions

`CDPracticeSession` records practice with:

- Quality metric (rating)
- Duration
- Method (independent, partnered, etc.)
- Partnership tracking for group practice

PDF Rendering

`WorkPDFRenderer` generates printable PDFs of work items — useful for physical classroom records.

Presentations Module

Directory: `Presentations/`

The feature is organized into `Assignments/`, `Planning/`, `Overview/`, `Workflow/`, and `Services/`.

Scheduling Calendar

Visual calendar for planning when to present lessons. The blocking algorithm prevents over-scheduling by checking:

- How many presentations are already scheduled for a day
- Whether prerequisite lessons have been completed

State Machine

```
draft → scheduled (set scheduledFor date)
scheduled → presented (set presentedAt, freeze snapshots)
draft → presented (skip scheduling, direct recording)
```

Mastery Tracking

CDLessonPresentation tracks per-student mastery:

```
presented → practicing → readyForAssessment → proficient
```

masteredAt records when proficiency was achieved.

Attendance Module

Directory: **Attendance/**

- **Daily grid** — mark present/tardy/absent per student
- **Monthly view** — calendar heatmap of attendance
- **Tardy reports** — analytics on tardiness patterns
- **Email** — send attendance summaries to families

Planning Module

Directory: **Planning/**

Multiple planning views:

View	Purpose
Agenda	Drag-and-drop lesson scheduler
Checklist	Class-wide subject tracking
Open Work	All active work items
Projects	Project-based learning management
Classroom Jobs	Job rotation management

Notes Module

Across: **Models/NoteEntity.swift**, **Notes/Observations/**, **Notes/Editor/**, **Notes/QuickCapture/**

Universal observation system. Notes can be:

- Scoped to all students, one student, or multiple students
- Categorized (academic, behavioral, social, emotional, health, attendance, general)
- Tagged with custom tags (format: "**TagName|Color**")
- Pinned, flagged for follow-up, or marked for inclusion in reports
- Attached to work items via the **unifiedNotes** relationship

Todos Module

Across: `Models/ToDoItemEntity.swift`, `Todos/Views/`, `Todos/Support/`, `Todos/Services/`

Teacher task management with:

- **Subtasks** (`CDToDoSubtask`) — checklist within a todo
- **Tags** — categorization
- **Due dates** — with overdue tracking
- **Export** (`ToDoExportService`) — export to files

Settings Module

Directory: `Settings/`

Section	What it configures
School Calendar	Non-school days, overrides, schedule templates
Age Levels	Lower/upper elementary configuration
Reminders	EventKit sync (macOS)
Calendar	Calendar display settings
AI/Chat	Private Cloud Compute opt-in; lesson-planning prompt, temperature and timeout
CloudKit	Sync enable/disable, status, deduplication
Data	Statistics, export, import, test student filtering
Templates	Meeting and note templates

Backup & Restore

Directory: `Backup/`

Service	Role
<code>BackupService</code>	Basic backup creation/restoration
<code>EnhancedBackupService</code>	Streaming, incremental, encryption
<code>BackupTransactionManager</code>	Atomic operations
<code>SelectiveRestoreService</code>	Restore specific data subsets
<code>CloudBackupService</code>	iCloud backup management

Service	Role
BackupSharingService	Share backups between devices/users
SelectiveExportService	Export specific model types
BackupValidationService	Verify backup integrity
ChecksumVerificationService	Hash verification

AI Chat

Directory: `Chat/` (`Chat/Services/` contains chat orchestration)

Chat with Apple Intelligence about classroom data (see `AIClientRouter` above). The system:

1. Builds a classroom snapshot (student names, lesson counts, recent activity)
2. Enriches each question with relevant context (Tier 2)
3. Maintains conversation history (trimmed to token budget)
4. Streams responses in real-time

Additional AI services:

- `StudentAnalysisService` — progress analysis per student
- `LessonPlanningService` — AI-assisted lesson planning
- `ReportGeneratorService` — generate progress reports

Part 9: Utilities & Extensions

Safe Fetch Pattern

File: `Utils/NSManagedObjectContext+SafeFetch.swift`

The most important utility in the app. Every database query should use these:

```
// Returns empty array on error (never throws)
context.safeFetch(request) -> [T]

// Returns nil on error or not found
context.safeFetchFirst(request) -> T?

// Throws but deduplicates automatically (CloudKit fix)
context.fetchUnique(request) -> [T]
```

`fetchUnique` uses the `.uniqueByID` collection extension to handle CloudKit-created duplicates.

Collection Extensions

File: `Utils/Collection+Extensions.swift`

```
// Create lookup dictionary from identifiable array
[CDStudent].dictionaryByID() -> [UUID: CDStudent]

// More readable emptiness check
array.isEmpty -> Bool

// Split array by predicate
array.partitioned(by: predicate) -> (matching: [T], rest: [T])
```

Date Utilities

Files: `Utils/Date+Normalization.swift`, `Utils/DateCalculations.swift`,
`Utils/DateFormatters.swift`

- `AppCalendar.startOfDay(date)` — normalized date used everywhere for date comparisons
- `AppCalendar.shared` — immutable Gregorian calendar on `TimeZone.autoupdatingCurrent`; the canon behind `\.calendar` and every non-view date computation
- Date formatters shared as static instances to avoid re-creation

String Utilities

Files: `Utils/String+Extensions.swift`, `Utils/String+FuzzyMatch.swift`,
`Utils/StringNormalization.swift`

- `.trimmed()` — removes leading/trailing whitespace
- Fuzzy matching for lesson name search in the command bar
- String normalization for consistent sorting

Other Utilities

File	Purpose
<code>Array+SafeAccess.swift</code>	Bounds-checked array access
<code>NSManagedObjectContext+SafeSave.swift</code>	Safe save with error handling
<code>PredicateHelpers.swift</code>	Common NSPredicate builders
<code>ValidationHelpers.swift</code>	Input validation
<code>CSVUtils.swift</code>	CSV parsing and generation
<code>AgeUtils.swift</code>	Age calculation from birthday
<code>MarkdownExporter.swift</code>	Export data as Markdown
<code>Double+Formatting.swift</code>	Number formatting
<code>View+ConditionalModifiers.swift</code>	Platform-specific view modifiers
<code>View+PlatformStyles.swift</code>	iOS vs macOS styling
<code>SyncedPreferencesStore.swift</code>	iCloud-synced preferences
<code>Logger+Extensions.swift</code>	Structured logging categories
<code>PerformanceLogger.swift</code>	Stutter detection (>100ms frames)

Part 10: How-To Reference

Adding a New Entity

1. **Add the entity in the Core Data model editor** (`CosmicDaybook.xcdatamodeld`) with attributes.
2. **Create the `NSManagedObject` subclass** in the appropriate feature directory or `Models/`:

```
public class CDMyNewEntity: NSManagedObject {
    @NSManaged public var id: UUID
    @NSManaged public var name: String
    @NSManaged public var statusRaw: String
    @NSManaged public var studentID: String
    @NSManaged public var modifiedAt: Date

    // Computed enum access
    var status: MyStatus {
        get { MyStatus(rawValue: statusRaw) ?? .active }
        set { statusRaw = newValue.rawValue }
    }

    // Core Data creation pattern
    static func create(in context: NSManagedObjectContext) -> CDMyNewEntity {
        let entity = CDMyNewEntity(context: context)
        entity.id = UUID()
        entity.name = ""
        entity.statusRaw = "active"
        entity.studentID = ""
        entity.modifiedAt = Date()
        return entity
    }
}
```

2. **Follow CloudKit rules:** string UUIDs, raw enums, no unique constraints, `modifiedAt` timestamp.
3. **Add indexes** for frequently queried fields in the model editor.
4. **Register in the Core Data model** — add the entity to `CosmicDaybook.xcdatamodeld`.
5. **Add a lightweight migration** if modifying an existing entity's schema.

Creating a Repository

1. **Create the repository file** in `Repositories/`:

```

@MainActor struct MyNewRepository: SavingRepository {
    typealias Entity = CDMyNewEntity
    let context: NSManagedObjectContext
    let saveCoordinator: SaveCoordinator?

    func fetchAll() -> [CDMyNewEntity] {
        let request: NSFetchedRequest<CDMyNewEntity> = CDMyNewEntity.fetchRequest()
        request.sortDescriptors = [NSSortDescriptor(keyPath: \CDMyNewEntity.name, ascending:
true)]
        return context.safeFetch(request)
    }

    func fetch(id: UUID) -> CDMyNewEntity? {
        let request: NSFetchedRequest<CDMyNewEntity> = CDMyNewEntity.fetchRequest()
        request.predicate = NSPredicate(format: "id == %@", id as CVarArg)
        request.fetchLimit = 1
        return context.safeFetchFirst(request)
    }

    func save(reason: String? = nil) -> Bool {
        if let coordinator = saveCoordinator {
            return coordinator.save(context, reason: reason)
        }
        return (try? context.save()) != nil
    }
}

```

2. **Construct it where it is used**, passing the caller's context and save coordinator.

Adding a Service

1. **Create the service file** in **Services/**:

```

@MainActor struct MyNewService {
    let context: NSManagedObjectContext

    func doSomething(...) -> Result {
        // Business logic here
        // Use safeFetch for queries
        // Use SaveCoordinator for saves
    }
}

```

2. **Register in AppDependencies**:

```

private var _myNewService: MyNewService?
var myNewService: MyNewService {
    if _myNewService == nil {
        _myNewService = MyNewService(context: viewContext)
    }
    return _myNewService!
}

```

3. **Use in views**:

```

@Environment(\.dependencies) private var dependencies
// ...
let result = dependencies.myNewService.doSomething(...)

```

Building a New Screen

1. Create a ViewModel (if needed):

```
@Observable @MainActor final class MyScreenViewModel {
    private let context: NSManagedObjectContext

    // Outputs
    private(set) var items: [CDMyEntity] = []

    init(context: NSManagedObjectContext) {
        self.context = context
    }

    func reload() {
        let request: NSFetchedRequest<CDMyEntity> = CDMyEntity.fetchRequest()
        items = context.safeFetch(request)
    }
}
```

2. Create the View:

```
struct MyScreenView: View {
    @Environment(\.managedObjectContext) private var context
    @Environment(\.dependencies) private var dependencies
    @State private var viewModel: MyScreenViewModel?

    var body: some View {
        List { ... }
        .task {
            if viewModel == nil {
                viewModel = MyScreenViewModel(context: context)
            }
            viewModel?.reload()
        }
    }
}
```

3. For simple lists, skip the ViewModel and use `@FetchRequest`:

```
struct SimpleListView: View {
    @FetchRequest(sortDescriptors: [NSSortDescriptor(keyPath: \CDMyEntity.name, ascending: true)])
    private var items: FetchedResults<CDMyEntity>

    var body: some View {
        List(items) { item in ... }
    }
}
```

4. Register the navigation route: add the case to `RootView+NavigationItem` (the display name, icon and accessibility hint switches are exhaustive), place it in exactly one group in `RootView+NavigationGroup`, route it in `RootDetailContent`, and pin its raw value in `NavigationGroupTests.rawValueArePinned`. Never rename an existing raw value.

Adding a Command Bar Intent

1. Add the intent to `RecordIntent` in `CommandBarTypes.swift`:

```
case myNewIntent // keyword triggers
```

2. Add trigger keywords in `LocalCommandParser.swift`:

```
// In the keyword families section  
case "mytrigger", "myword": return .myNewIntent
```

3. Add the action to `CommandAction`:

```
case myNewAction(param: Type)
```

4. Handle the action in `CommandBarViewModel`.

CloudKit Considerations Checklist

When modifying any model or adding new features:

- [] UUIDs stored as `String`, not `UUID`
 - [] Enums stored as `*Raw: String` with computed property
 - [] No unique constraints in the Core Data model
 - [] `modifiedAt: Date` field present
 - [] Foreign keys are `String`, not Core Data relationships
 - [] Large data uses "Allows External Storage" in the model editor
 - [] Arrays use JSON encoding (e.g., `CloudKitStringArrayStorage`)
 - [] Deduplication handled gracefully (use `fetchUnique` or `uniqueByID`)
-

Part 11: Configuration & Debugging

CloudKit Configuration

- **Container:** `iCloud.DanielSDeBerry.MariasNoteBook`
- **Default:** Enabled (can be toggled in Settings)
- **Auto-disabled:** During XCTest runs
- **Sync strategy:** Automatic with `modifiedAt` last-writer-wins
- **Deduplication:** Periodic cleanup via `deduplicateAllEntities()`
- **Health monitoring:** `CloudKitSyncStatusService` provides real-time status

EventKit Permissions (macOS)

- Reminder sync requires user permission
- `ReminderSyncService` handles permission requests
- Calendar access separate from reminder access
- Conditional compilation: only available on macOS

API Keys

The app's AI is Apple Intelligence only and needs no API key. (Claude and OpenAI keys were removed on 2026-09-26.)

Performance Monitoring

- **Stutter detection:** `PerformanceLogger` flags frames >100ms
- **SQLite logging:** Enabled in Debug builds via environment variables
- **Memory pressure:** `MemoryPressureMonitor` observes system memory and proactively clears caches
- **Migration diagnostics:** Logged to console, accessible in Settings

Debugging Aids

Tool	Purpose
<code>Logger.app(category:)</code>	Structured logging with categories

Tool	Purpose
<code>PerformanceLogger</code>	Frame timing and stutter detection
<code>SyncEventLogger</code>	CloudKit event logging
Settings > Data	Database statistics, export, integrity checks
Console.app	Filter by <code>CosmicDaybook</code> subsystem

Build Configuration

Requirement	Value
Xcode	16.0+
Swift	6.0+
iOS	26.0+
macOS	26.0+
Frameworks	SwiftUI, Core Data (NSPersistentCloudKitContainer), CloudKit (optional), EventKit (macOS)

Code Standards

- **SwiftLint** enforced (0 violations)
- `@MainActor` on all ViewModels and UI-touching services
- `private(set)` for ViewModel published properties
- **Composition over inheritance** throughout
- **Safe operations** via extensions (`safeFetch`, `safeSave`)

Appendix A: File Directory Map

```
Cosmic Daybook/
+-- AppCore/
|   +-- CosmicDaybookApp.swift      App entry point
|   +-- AppBootstrapper.swift      Startup state machine
|   +-- AppBootstrapping.swift     Synchronous init
|   +-- AppDependencies.swift      DI container
|   +-- AppRouter.swift            Navigation coordinator
|   +-- rootView.swift             Main shell
|   +-- rootView+NavigationItem.swift  NavigationItem, aliases, selection restore
|   +-- rootView+NavigationGroup.swift The sidebar / tab grouping table
|   +-- rootView/
|       +-- PieMenu.swift          Floating action menu
|       +-- QuickNoteGlassButton.swift Floating button
|       +-- RootSidebar.swift      macOS / visionOS sidebar
|       +-- RootAdaptiveTabs.swift iPhone tab bar / iPad sidebar
|       +-- RootDetailContent.swift Content routing
|
+-- Models/
|   +-- NoteEntity.swift           Universal observation (CDNote)
|   +-- Presentation.swift         Lesson scheduling (CDLessonAssignment)
|   +-- SupplyEntity.swift         Classroom inventory (CDSupply)
|   +-- (other shared entity files)
|
+-- Students/
|   +-- Models/StudentEntity.swift Student entity (CDStudent)
|   +-- Roster/                   Student list and cards
|   +-- Detail/                   Student detail tabs and state
|   +-- Files/                    Student documents
|   +-- Import/                   CSV import workflow
|   +-- Notes/                    Student observation timeline
|   +-- Presentations/            Student presentation workflow
|   +-- Progress/                 Curriculum progress and tracks
|   +-- Selection/                Student picker UI
|   +-- Meetings/                 One-on-one meetings
|   |   +-- Services/             Meeting insight generation
|   +-- Insights/Services/        Student analysis
|   +-- Recall/Services/          Recall scoring and retention
|   +-- YearPlan/Services/        Year-plan promotion
|
+-- Lessons/
|   +-- LessonEntity.swift         Lesson entity (CDLesson)
|   +-- LessonsListView.swift      Library view
|   +-- LessonDetailView.swift     Detail view
|   +-- ViewModels/GiveLessonViewModel.swift Lesson picker state
|   +-- Checklist/ViewModels/      Class-area checklist state
|
+-- Work/
|   +-- Models/WorkModelEntity.swift Work entity (CDWorkModel)
|   +-- Agenda/                   Work calendar and planning
|   +-- CheckIns/                 Progress check-ins
|   +-- Completion/               Completion records and rules
|   +-- Detail/                   Work detail screen and state
|   +-- Practice/                 Practice sessions
|   +-- SampleWork/               Sample-work capture
|   +-- Steps/                    Work-step editing
```

	+++ Support/	Scheduling, filtering, and data access
	+++ Services/	Lifecycle and PDF rendering
	+++ WorkCard/	Shared work card presentations
+++	Presentations/	Scheduling and recording
	+++ Assignments/	Assignment detail and history
	+++ Planning/	Scheduling and blocking
	+++ Overview/	Presentation queues and filters
	+++ Workflow/	Presentation and follow-up workflow
	+++ Services/	Presentation splitting
+++	Attendance/	Daily tracking
+++	Planning/	Scheduling and curriculum
	+++ AIPlanning/LessonPlanning/	AI planning service and state
	+++ Services/	Next-lesson planning
+++	Inbox/	Follow-up management
	+++ Services/	Follow-up categorization
+++	Today/	Daily hub
	+++ Views/	Today screen and sections
	+++ ViewModels/TodayViewModel.swift	Today hub state
	+++ Support/	Builders, loaders, caches, navigation
+++	Todos/	Todo management
	+++ Views/	Screens, forms, rows, and sheets
	+++ Support/	Filters, sorting, and date parsing
	+++ Services/	Parsing, notifications, and export
+++	Projects/	Project-based learning
	+++ Services/	Session work assignment
+++	Notes/	Observation and note capture
	+++ Observations/	Browsing, filtering, and AI summaries
	+++ Editor/	Note editors and tag selection
	+++ QuickCapture/	Quick-note sheet and parsing
	+++ Views/	Scoped note presentation
+++	Chat/Services/	AI chat orchestration
+++	Parsha/Services/	Parsha metadata and suggestions
+++	Procedures/Services/	Procedure operations
+++	Supplies/	Inventory management
	+++ Services/	Supply operations
+++	Topics/	
	+++ ViewModels/TopicDetailViewModel.swift	Topic detail state
+++	Settings/	App configuration
+++	Backup/	Backup and restore
+++	Services/	
	+++ CloudKitSyncStatusService.swift	Sync monitoring
	+++ CommandBar/	Natural language parsing
	+++ (cross-feature infrastructure and system integrations)	
+++	Repositories/	
	+++ StudentRepository.swift	
	+++ LessonRepository.swift	
	+++ (one struct per entity family)	
+++	ViewModels/	
	+++ CommandBarViewModel.swift	Command bar state
+++	Components/	Reusable UI primitives
+++	Utils/	57+ utility files

Appendix B: Glossary

Term	Meaning
Presentation	The act of presenting a Montessori lesson to a student. Represented by <code>CDLessonAssignment</code> .
Follow-up	Work or re-presentation needed after an initial presentation.
Check-in	A progress note on an active work item (<code>CDWorkCheckIn</code>).
Track	A curriculum progression path — a sequence of steps a student moves through.
Great Lesson	One of the five Montessori "cosmic education" stories that frame the curriculum.
Album	A Montessori teacher's collection of lesson plans, organized by subject.
Three-period lesson	A Montessori teaching technique: introduction, recognition, recall.
Scope (Note)	Who a note applies to: all students, one student, or a specific group.
Raw enum	A string-stored enum value for CloudKit compatibility (e.g., <code>statusRaw</code>).
Safe fetch	A query wrapper that returns empty results instead of throwing on error.
Pie Menu	The circular action menu that appears on long-press of the floating button.
Command Bar	Natural language input for quick actions (presentations, work, notes, todos).